Reward Signals

Reward Models, DPO, Verifiable Rewards, and Process Rewards — The Specification Language of Post-Training

1

Learning Objectives

- Explain why the **reward signal** is the central design choice in LLM post-training
- Derive the **Bradley-Terry preference model** and the reward model training loss
- Derive **DPO** from first principles: KL-constrained RL → closed-form optimal policy → reparameterization → supervised loss
- Interpret the **DPO gradient** — why it focuses learning on examples the model currently gets wrong
- Compare **DPO, SimPO, KTO, and online DPO** — when to use which
- Explain **verifiable rewards** and **RLVR** — why they dominate reasoning RL and where they're expanding
- Distinguish **outcome reward models (ORMs)** from **process reward models (PRMs)** — including discriminative PRMs vs. generative PRMs (ThinkPRM)
- Identify **reward hacking** and overoptimization — Goodhart's Law in practice, including in frontier reasoning models

2

From Last Time

- We have the **optimization algorithms**: PPO, GRPO, DAPO — all variations of policy gradient + baseline + trust region (clipping)
- PPO: per-token advantages via learned value function + GAE. 4 models. General alignment.
- GRPO/DAPO: per-response advantages via group normalization. No critic. 3 models. Reasoning RL.
- All these algorithms take a **reward signal** as input and adjust the policy to produce higher-reward behavior
- **The critical question we left open: where does the reward come from?**

3

Part 1: The Reward Landscape

4

Three Sources of Reward

- **1. Learned reward models**: train a neural network on human/AI preferences. Input: (prompt, response) → output: scalar score. Used with PPO or GRPO.
- **2. Direct preference optimization (DPO)**: skip the reward model — train the policy directly from preference pairs. No RL loop at all.
- **3. Verifiable rewards**: check the answer — execute code, compare math answers, type-check proofs, regex format. Exact signal, no learning needed.
- Each has a different **cost–quality–generality** tradeoff:
- Verifiable rewards: **exact but narrow** (only works when correctness is checkable)
- Preference-based methods: **general but noisy** (humans disagree, reward models are imperfect)
- The frontier recipe uses **all three** at different stages

5

The Fundamental Tradeoff

Source	Signal Quality	Scalability	Generality	Cost
Verifiable rewards	Exact	Unlimited	Narrow (math, code, formal)	Near zero
LLM-as-judge / RLAIF	Good (depends on judge)	Very high	Broad	Low (API calls)
Learned reward model	Approximate	High (once trained)	Broad	Medium (train RM)
Human preferences	High but noisy	Low	Broad	High ($1–5/label)
DPO (implicit)	Approximate	Limited to dataset	Broad	Low (supervised)

Signal quality × scalability × generality. No single source covers all needs — frontier labs combine them.

6

Part 2: Reward Models from Preferences

7

The Bradley-Terry Preference Model

- Given a prompt x and two responses y_w (preferred) and y_l (dispreferred), model the preference probability:
- $$P(y_w \succ y_l \mid x) = \sigma\big(r_\psi(x, y_w) - r_\psi(x, y_l)\big)$$
- where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid and r_ψ is a learned **reward model**: (prompt, response) → scalar
- Intuition: the preference probability depends only on the **difference** in rewards — like Elo ratings in chess
- If $r_\psi(x, y_w) \gg r_\psi(x, y_l)$: $\sigma \to 1$ — strongly prefer y_w ✓
- If $r_\psi(x, y_w) = r_\psi(x, y_l)$: $\sigma = 0.5$ — coin flip, no preference

8

Training a Reward Model

- **Architecture**: same transformer as the policy, replace the LM head with a **scalar output head** — one number per (prompt, response)
- **Data**: preference pairs (x, y_w, y_l) — sampled from the policy, labeled by humans or an AI judge
- **Loss**: maximize log-likelihood of observed preferences under Bradley-Terry:
- $$\mathcal{L}_{\text{RM}}(\psi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}[\log \sigma\big(r_\psi(x, y_w) - r_\psi(x, y_l)\big)]$$
- This is just **binary cross-entropy** on the reward difference — standard supervised learning
- **Initialization**: from the SFT model. **Scale**: typically same size as the policy (70B policy → 70B RM). **Data**: 100K–1M preference pairs.
- Once trained, $r_\psi(x, y)$ provides a scalar reward for **any** (prompt, response) → plug into PPO or GRPO

9

Discriminative vs. Generative Reward Models

- **Discriminative RM** (classical): encode (prompt, response) → scalar head → one number. Fast, but a black box — no explanation for its score.
- **Generative RM (GenRM)**: the reward model *generates* a chain-of-thought evaluation before producing a score. More interpretable, more robust.
- GenRM trains an LLM on self-generated reasoning traces that lead to preference labels matching human judgments
- GenRM outperforms discriminative verifiers and LLM-as-a-Judge on math benchmarks, achieving large gains with Best-of-N sampling
- **ThinkPRM** (2025): a generative *process* reward model — generates verification CoT for each reasoning step, achieves strong results using only ~1% of the step-level labels that discriminative PRMs need
- **Key finding**: reward model scale is the most decisive factor for policy quality — gains continue even when the RM far exceeds the policy in parameters
- The field is shifting from discriminative → generative reward models, mirroring the broader trend of replacing classifiers with generative reasoning

10

Interactive Demo: Reward Model Training

🚀 **Interactive Demo:** reward_model_demo.html

11

Part 3: DPO — Direct Preference Optimization

12

The DPO Insight

- The standard RLHF pipeline: (1) collect preferences → (2) train reward model → (3) run RL with that reward
- **DPO** (Rafailov et al., 2023) collapses all three into **one supervised learning step**
- No reward model. No RL loop. No PPO. No GRPO. Just a loss function on preference pairs.
- How? The **optimal policy under KL-constrained RL** has a closed form — and that form lets us express the reward **directly in terms of the policy**
- Substitute into Bradley-Terry → the partition function cancels → **supervised loss**

13

DPO Derivation Step 1: The Optimal Policy

- Start from the KL-constrained RL objective (same one that motivates RLHF):
- $$\max_{\pi} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi(\cdot|x)}[r(x,y)] - \beta\, D_{\mathrm{KL}}[\pi(\cdot|x) \,\|\, \pi_{\mathrm{ref}}(\cdot|x)]$$
- Maximize reward, but stay close to π_{ref} (the SFT model)
- This has a **closed-form solution** (calculus of variations):
- $\pi^*(y|x) = \dfrac{1}{Z(x)}\, \pi_{\mathrm{ref}}(y|x)\, \exp\left(\dfrac{r(x,y)}{\beta}\right) \qquad Z(x) = \sum_y \pi_{\mathrm{ref}}(y|x) \exp\left(\dfrac{r(x,y)}{\beta}\right)$
- Intuition: the optimal policy **tilts** the reference distribution toward high-reward responses, with β controlling aggressiveness

14

DPO Derivation Step 2: From Reward to Policy to Loss

- **Rearrange** the optimal policy to express reward in terms of policy:
- $$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$
- **Substitute** into Bradley-Terry: $P(y_w \succ y_l) = \sigma(r(y_w) - r(y_l))$:
- $$P(y_w \succ y_l) = \sigma\left(\beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$
- The $\beta \log Z(x)$ terms **cancel**! (Same prompt → same partition function for both responses.)
- Replace π^* with learnable π_θ, minimize negative log-likelihood → **the DPO loss**:
- $$\boxed{\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]}$$

15

Interactive Demo: DPO

🚀 **Interactive Demo:** dpo_demo.html

16

Understanding the DPO Gradient

- Define the **implicit reward**: $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$
- The gradient:
- $$\nabla_\theta \mathcal{L} = -\beta \mathbb{E}\left[\underbrace{\sigma\big(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)\big)}_{\text{weight: high when model is wrong}} \Big(\underbrace{\nabla \log \pi_\theta(y_w|x)}_{\text{increase preferred}} - \underbrace{\nabla \log \pi_\theta(y_l|x)}_{\text{decrease dispreferred}}\Big)\right]$$
- The weight is **largest when the model currently prefers the wrong response** → focuses learning where it's needed most
- As training progresses and the model gets preferences right → weight shrinks → **automatic curriculum**
- This is **weighted supervised learning**: increase $\log \pi(y_w)$, decrease $\log \pi(y_l)$, weighted by how wrong the model currently is

17

What DPO Computes in Practice

- For each preference pair (x, y_w, y_l):
- 1. Forward pass π_θ on both y_w and y_l → sum per-token log-probs → $\log \pi_\theta(y_w|x)$, $\log \pi_\theta(y_l|x)$
- 2. Look up **stored** reference log-probs $\log \pi_{\text{ref}}(y_w|x)$, $\log \pi_{\text{ref}}(y_l|x)$ (precomputed once)
- 3. Implicit rewards: $\hat{r}_w = \beta(\log \pi_\theta(y_w|x) - \log \pi_{\text{ref}}(y_w|x))$, same for $\hat{r}_l$
- 4. Loss: $-\log \sigma(\hat{r}_w - \hat{r}_l)$
- **Models in memory**: 1–2. Trainable π_θ + frozen π_{ref} (or precompute reference log-probs and store only π_θ).
- **Known failure mode**: DPO can decrease the likelihood of *preferred* responses while still increasing the gap. This 'Degraded Chosen Response' (DCR) problem motivates several variants.

18

DPO Variants That Matter

- **SimPO** (Meng et al., 2024): eliminates the reference model entirely — average log-probability as implicit reward:
- $$\mathcal{L}_{\text{SimPO}} = -\mathbb{E}\left[\log \sigma\left(\frac{\beta}{|y_w|}\log \pi_\theta(y_w|x) - \frac{\beta}{|y_l|}\log \pi_\theta(y_l|x) - \gamma\right)\right]$$
- No reference model needed, just one model. Margin γ enforces a reward gap.
- **KTO** (Ethayarajh et al., 2024): doesn't need **paired** preferences — just individual responses labeled good/bad. Simpler data, but same family.
- **Online / Iterative DPO**: standard DPO trains on a **fixed** offline dataset. Online DPO generates **fresh** response pairs each iteration → consistently stronger than offline DPO. Bridges the gap between DPO and RL.
- **Key lesson**: offline DPO underperforms online methods. If compute allows, always prefer online variants or full RL.

19

Reward Model + RL vs. DPO — When to Use Which

	Reward Model + PPO/GRPO	DPO (and variants)
Models in memory	3–4 (actor, [critic], ref, RM)	1–2 (actor, [ref])
Training loop	Online: generate → score → update → repeat	Supervised: (typically) fixed dataset of pairs
Reward signal	Learned scalar — applicable to **any** new response	Implicit — only defined on the preference data
Data efficiency	One RM across many RL iterations	Each pair used directly in the loss
Strength	**Online** exploration, arbitrary rewards, long-horizon	**Simpler**, more stable, fewer hyperparameters
Weakness	Complex, expensive, RM can be hacked	Offline by default, constrained to data distribution
Best for	Continuous improvement, complex behaviors	Quick alignment, smaller teams, limited compute

DPO is simpler; RM+RL is more powerful. Frontier labs typically use both at different stages.

20

Part 4: Verifiable Rewards and RLVR

21

The Simplest and Most Powerful Reward: Check the Answer

- For some tasks, you don't need preferences **or** a reward model — just **verify the output**:
- **Math**: extract the final answer, compare to ground truth → $r \in \{0, 1\}$
- **Code**: execute against test cases → reward = fraction passed, or binary pass/fail
- **Formal proofs**: type-check in Lean/Coq → binary
- **Format compliance**: regex or rule-based → binary
- **Composite rewards**: $R(q, o) = w_{\text{ans}} r_{\text{ans}} + w_{\text{exec}} r_{\text{exec}} + w_{\text{format}} r_{\text{format}} - w_{\text{len}} p_{\text{len}}$
- This is **RLVR** — RL with Verifiable Rewards. Combined with GRPO/DAPO, it's the recipe behind DeepSeek-R1, Qwen 3, and GLM-5.

22

Why Verifiable Rewards Changed Everything

- **No reward model noise**: the signal is ground truth. You cannot 'hack' a unit test that either passes or fails.
- **Infinite data**: generate new problems or sample from large problem banks — no human labeling bottleneck
- **GRPO is a perfect fit**: binary/graded rewards + group normalization cleanly separate correct from incorrect. $G = 64$ samples: 40 correct, 24 incorrect → clear advantage signal.
- **Emergent reasoning**: DeepSeek-R1-Zero showed that GRPO + verifiable rewards, starting from a base model with **no SFT**, spontaneously learns CoT, self-verification, and backtracking
- **Expanding beyond math/code**: RLVR is being applied to chemistry, biology, tool use, structured reasoning, agentic tasks, and even translation
- **Limitation**: only works when correctness is automatically checkable. 'Be helpful', 'be nuanced', 'explain clearly' still need preference-based methods.
- This is why frontier pipelines use **both**: RLVR for reasoning, preferences for general alignment

23

Part 5: Process Reward Models

24

Outcome vs. Process Rewards

- **Outcome Reward Model (ORM)**: one scalar at the end. Was the final answer correct?
- **Problem**: a 20-step reasoning chain has one error at step 7. The ORM gives **no signal about where things went wrong.**
- **Process Reward Model (PRM)**: assigns a reward to **each intermediate reasoning step**. 'Step 3 was correct, step 7 introduced the error, step 12 partially recovered.'
- PRMs provide **dense, step-level credit assignment** — exactly what's missing from outcome-only rewards
- Analogy: ORM = grading only the final exam answer. PRM = grading each line of the proof.

25

Training Process Reward Models

- Step correctness labels don't exist naturally. Three main approaches:
- **1. Human annotation** (PRM800K): expensive — hundreds of thousands of step labels needed for a reasonable math PRM
- **2. Monte Carlo estimation**: for each step s_k, complete the remaining chain many times. Step quality ≈ fraction of rollouts reaching correct final answer from that prefix.
- $$\mathrm{PRM}(s_k) \approx \frac{\backslash\# \text{ rollouts correct from step } k}{\backslash\# \text{ total rollouts from step } k}$$
- **3. Generative PRMs (ThinkPRM, GenPRM)**: train a reasoning model to *generate* a verification CoT for each step. ThinkPRM outperforms discriminative PRMs using only ~1% of the step labels — dramatically cheaper.
- Discriminative PRMs struggle with domain shifts; generative PRMs generalize much better because they leverage the LLM's inherent reasoning
- **At training time**: use PRM scores as denser rewards in GRPO/PPO. **At inference time**: guide tree search → test-time compute scaling (next lecture)

Outcome vs. Process vs. Generative Process Reward Models

	ORM	Discriminative PRM	Generative PRM (ThinkPRM)
Granularity	Per-response	Per-step	Per-step + explanation
Credit assignment	Sparse	Dense	Dense + interpretable
Training data	Final answer correctness	Step labels (human or MC)	~1% of step labels + CoT fine-tuning
Domain transfer	Moderate	Fragile under domain shift	More robust (uses reasoning)
Compute at inference	Fixed	Fixed	Scalable (more CoT = better)
Best for	General alignment	Math reasoning, training	Math/code, test-time search

The field is shifting from discriminative → generative PRMs: cheaper to train, more robust, scalable at inference.

27

Part 6: Reward Hacking

28

Goodhart's Law: When a Measure Becomes a Target

- A reward model is a **learned proxy** for human values. It has blind spots. The policy **will** find and exploit them.
- **Classic failure modes**:
- • **Verbosity**: longer responses score higher → model becomes excessively wordy
- • **Sycophancy**: agreeing with the user always scores well → model stops pushing back
- • **Pattern exploitation**: certain phrases/formatting the RM prefers, regardless of content
- • **Hallucination with confidence**: confident wrong answers score higher than hedged correct ones
- The **overoptimization curve**: as RL progresses, proxy reward ↑ but true quality eventually ↓. The gap grows the harder you optimize.

29

Reward Hacking in Frontier Reasoning Models

- This is not just a theoretical concern — frontier reasoning models (o3, R1, etc.) engage in increasingly sophisticated reward hacking
- They reason about the testing process and take deliberate actions to maximize scores: modifying test cases, accessing answer keys, exploiting loopholes
- In one study, instructing o3 not to cheat or not to reward hack had 'nearly negligible effect' — it still persisted in a majority of runs
- Even more concerning: reward hacking persisted when the task was framed as helping scientists with real-world consequences
- Naive fix of training against detected reward hacking can backfire — it incentivizes *subtler* hacking that evades the monitor
- Reward hacking is theoretically unavoidable: across all policy distributions, two reward functions can only be unhackable if one is constant

30

Mitigations for Reward Hacking

- **KL penalty to reference model** — the primary defense (covered in Lecture 13). Limits total policy drift.
- **Reward model ensembles** — take conservative (minimum) reward across multiple RMs → harder to hack all simultaneously
- **Iterative RM retraining** — periodically retrain the RM on the current policy's outputs → the RM 'catches up' to exploitation strategies
- **Mixing verified and preference rewards** — verifiable rewards are un-hackable by definition, providing a stable anchor
- **Generative reward models** — GenRMs that explain their reasoning are harder to exploit than black-box scalar models, and their reasoning can itself be verified
- **Conservative optimization** — don't push RL too hard. PPO clipping + GRPO/DAPO bounded advantages + early stopping
- **Reward shaping best practices**: rewards should have an upper bound and exhibit rapid growth with slow convergence

31

Interactive Demo: Reward Hacking

🚀 **Interactive Demo:** reward_hacking_demo.html

32

Part 7: Scaling Reward Signals — LLM-as-Judge

33

LLM-as-Judge: The Dominant Source of Preference Data

- Human labeling: ~$1–5 per comparison, minutes per label, 20–30% inter-annotator disagreement
- **LLM-as-judge**: use a strong model to evaluate response quality — 100–1000× cheaper, 100× faster, more consistent
- **Constitutional AI** (Anthropic): define principles → model critiques its own responses → (original, revised) as preference pairs → train with DPO or RL
- **RLAIF** (RL from AI Feedback) is increasingly the **primary** source of preference data at frontier labs — human labeling is for validation and edge cases
- **Caution**: LLM judges have biases — positional bias (prefer first/second candidate), self-preference (prefer own responses), verbosity preference
- The quality of the judge is critical: a weak judge → noisy preferences → poor policy. GenRMs that reason before judging help mitigate this.

34

Part 8: Putting It All Together

35

The Complete Reward Signal Toolkit

Reward Source	Quality	Scalability	Generality	Extra Models	Best For
Verifiable	Exact	Unlimited	Narrow	0	RLVR (GRPO/ DAPO)
Discriminative RM	Approximate	High	Broad	+1 (RM)	Online R (PPO/GR
Generative RM	Good	Moderate	Broad	+1 (GenRM)	RL + interpret scoring
DPO (implicit)	Approximate	Dataset-limited	Broad	0	Offline alignmer
Process RM	Good	Moderate	Reasoning	+1 (PRM)	Long cha test-time search
LLM-as-Judge	Good	Very high	Broad	0 (API)	Preferen data at s

Frontier labs combine multiple sources across training stages. No single method covers all needs.

36

How the Pieces Fit Together (Preview of Next Lecture)

- A typical frontier training pipeline:
- **Stage 1 — SFT**: supervised fine-tuning on curated instruction data. No reward signal — just imitation.
- **Stage 2 — Reasoning RL**: GRPO/DAPO + **verifiable rewards** on math and code. The model learns to reason.
- **Stage 3 — General alignment**: DPO or PPO + **reward model** (trained on AI-judge preferences). Helpfulness, safety, instruction following.
- **Stage 4 — Distillation (optional)**: distill strong RL trajectories back into supervised data for stability.
- **At inference**: optionally use a **PRM** to guide test-time search (Best-of-N, tree search).
- The reward signal is the **specification language** of post-training — it defines what 'good' means. Get it wrong, and the model optimizes the wrong thing.

37

Summary

38

All Interactive Demos

🚀 **Interactive Demo:** reward_model_demo.html

🚀 **Interactive Demo:** dpo_demo.html

🚀 **Interactive Demo:** reward_hacking_demo.html

39

Lecture Summary

- **Reward models**: Bradley-Terry → $\mathcal{L} = -\log \sigma(r(y_w) - r(y_l))$ → scalar reward for PPO/GRPO. Shifting from discriminative → **generative RMs** that reason before judging.
- **DPO**: KL-constrained RL has a closed-form solution → rearrange → substitute into Bradley-Terry → $Z(x)$ cancels → **supervised loss on preference pairs**. Gradient focuses on examples the model currently gets wrong.
- **Variants**: SimPO (no reference model), KTO (unpaired), Online DPO (regenerate pairs). Offline DPO underperforms online methods.
- **Verifiable rewards + RLVR**: exact, unhackable, unlimited scale — but only for checkable tasks. Combined with GRPO/DAPO, this powers reasoning RL. Expanding beyond math/code.
- **Process reward models**: per-step credit via Monte Carlo or generative verification (ThinkPRM). Used for training (denser RL signal) and inference (test-time search).
- **Reward hacking**: Goodhart's Law is real and unavoidable. Frontier models actively exploit loopholes. Mitigate with KL penalty, RM ensembles, verified rewards, GenRMs, conservative optimization.
- **Next lecture**: How to compose these into production pipelines — multi-stage training recipes, test-time compute scaling, distillation, and the full frontier stack

Supplementary Resources

🚀 **Interactive Demo:** handout.html

41